A PRACTICAL GUIDE · TEAM BRIEFING

Evaluating Large Language Models

How do we know an AI system is **actually good** — and prove it?

LLM-as-a-Judge | Cohen's Kappa | Judge biases | Singapore AI Verify

THE PROBLEM

Why bother with evals?

01

Ship with confidence

"It looks good" doesn't scale past a handful of examples. Evals turn gut-feel into evidence.

02

Catch regressions

A prompt tweak or model swap can quietly break things. Evals are your unit tests for AI.

03

Compare fairly

Model A vs Model B, v1 vs v2 — only a repeatable eval lets you choose on facts, not vibes.

> Evals are the difference between "trust me" and "here's the number."

THE LANDSCAPE

Three ways to grade an answer

DETERMINISTIC

Code-based metrics

Exact match, F1, BLEU/ROUGE. Fast, free, repeatable — but only when answers are constrained to one (or a few) known-good targets.

HUMAN

Human review

People read and rate. The gold standard for quality — but slow, costly, and inconsistent.

MODEL-BASED

LLM-as-a-judge

A strong model grades the output. Scales like code, judges like a human — *mostly*.

The rest of this talk is about the **third** one — and how to keep it honest.

WHY NOT JUST USE CODE METRICS?

Exact match falls apart on real language

Question — *Capital of Australia?*

Reference: "Canberra"

Model answer	Exact match
"Canberra"	✅
"It’s Canberra."	❌
"The capital is Canberra."	❌
"Sydney"	❌

> Three of those answers are correct — and the metric only liked one of them.

For open-ended work — summaries, chat replies, code, advice — there are **many** good answers and infinite phrasings. String-matching can't see meaning.

WHY NOT JUST USE HUMANS?

Human eval is the gold standard — and a bottleneck

SLOW

Hours, not seconds

Rating 1,000 responses by hand stalls every experiment you want to run.

COSTLY

Expensive at scale

Quality raters cost real money — and you need them again for every new version.

NOISY

Inconsistent

Two people disagree; one person disagrees with themselves on a Friday afternoon.

> We want human-like judgement at code-like speed. Enter the LLM judge.

THE IDEA

Let a model be the grader

Give a capable LLM:

- the **task** and the **output** to grade
- a clear **rubric** (what "good" means)
- a request for a **score + reason**

It returns a structured verdict in seconds, for cents — across thousands of items.

JUDGE OUTPUT

```
{
  "score": 4,
  "criteria": {
    "correct": true,
    "helpful": true,
    "tone": "good"
  },
  "reason": "Accurate and clear;
   could add a backup step."
}
```

THREE FLAVOURS

How judges are asked to grade

POINTWISE

Score one answer

"Rate this 1–5 on helpfulness." Simple; good for tracking a metric over time.

PAIRWISE

Pick the winner

"Which is better, A or B?" Easier and more reliable than absolute scores — great for A/B.

REFERENCE

Grade vs a gold answer

"Does this match the reference?" Anchors the judge when you have a known-good answer.

Rule of thumb: humans (and LLMs) are **better at comparing** than at assigning absolute scores.

ANATOMY

What a good judge prompt contains

- **Role + task context** — what is being graded and why
- **An explicit rubric** — define each score level, not just "rate 1–5"
- **Concrete criteria** — correctness, helpfulness, safety, tone…
- **Reason before score** — make it explain first, then commit
- **Structured output** — JSON you can parse and aggregate

> Vague rubric in → noisy scores out. The rubric *is* the eval.

▸ **See a real judge, step by step**

THE CATCH

Judges are confident — and biased

POSITION BIAS

Order matters

In pairwise mode, judges tend to prefer whichever answer is shown *first* — regardless of quality.

VERBOSITY BIAS

Longer looks smarter

More words and more formatting get rated higher, even when they add nothing.

SELF-PREFERENCE

Likes its own kind

A model tends to favour text written in its own style — including its own outputs.

FORMAT BIAS

Bullets & bold win

Pretty formatting can sway the score independent of substance.

▸ **Flip the answer order & watch the verdict change**

THE CATCH · ONE BIAS, UP CLOSE

Format bias: styling ≠ substance

PLAIN PROSE	FORMATTED · SAME FACTS
Overall 3 / 5	**Overall 5 / 5**
All the right steps — in one dense paragraph.	Identical content with a heading, **bold**, and a numbered list.

Every fact is the same — same steps, same CSV/JSON, same 24-hour expiry. Only the **markdown** changed, yet the judge scored it two points higher.

The judge rewarded **presentation, not correctness**. Fix: instruct it to ignore formatting & length, or normalise formatting before grading — then watch the length/score correlation.

▸ **See the same answer score two ways**

MITIGATIONS

Keeping the judge honest

- **Randomise & swap order** — run A/B and B/A; only trust a winner that survives both
- **Anchor with references** — give a gold answer or few-shot examples to calibrate
- **Control for length** — instruct "ignore length"; sanity-check the length/score correlation
- **Use a different model as judge** — reduce self-preference
- **Ensemble & aggregate** — multiple judges or repeated runs, then take a majority/mean

> But every mitigation begs the real question: how do we know the judge is right at all?

VALIDATION

Treat the judge like a new hire

1. Take a **sample** of real outputs.
2. Have **humans** label them carefully.
3. Have the **judge** label the same items.
4. Measure **how often they agree**.

If the judge agrees with your experts, you can trust it to scale. If not, fix the rubric and repeat.

> The judge isn't trustworthy because it's an AI.
> It's trustworthy because it agrees with humans you trust.

A TRAP

"They agreed 80% of the time!" — so what?

Imagine grading pass/fail, where **90% of answers pass**.

Two raters who just say *"pass"* to everything — **ignoring the content entirely** — will agree about **80%+** of the time.

High raw agreement can be almost **entirely luck**, driven by the common answer.

> We need agreement beyond what chance alone would produce.

That correction is exactly what Cohen's Kappa gives us.

THE METRIC

Cohen’s Kappa, in one line

$$\kappa = (p_o - p_e) / (1 - p_e)$$

p_o

Observed agreement

How often the two raters actually agreed.

p_e

Expected by chance

How often they'd agree just by guessing at their base rates.

κ

The credit that's left

Agreement *above* chance, scaled so 1 = perfect, 0 = pure luck.

Read it as: **"of the agreement that wasn't guaranteed by luck, how much did they actually achieve?"**

WORKED EXAMPLE

Judge vs human on 100 answers

	Judge: PASS	Judge: FAIL	Total
Human: PASS	**45**	10	**55**
Human: FAIL	5	**40**	**45**
Total	**50**	**50**	**100**

Diagonals = they agreed (85 of 100). The **Total** row/column are each rater's base rates.

$p_o = (45 + 40) / 100 = \mathbf{0.85}$

$p_e = \mathbf{0.50}$ — how often they'd agree by luck:

Human says PASS 55%, Judge 50% → both PASS = 0.55 × 0.50 = 0.275.
Both FAIL = 0.45 × 0.50 = 0.225. Add them: **0.275 + 0.225 = 0.50**.

$\kappa = (0.85 - 0.50) / (1 - 0.50) =$ **0.70**

▸ **Try the live Kappa calculator**

INTERPRETING K

How good is "good enough"?

κ range	Landis & Koch label	Trust the judge?
< 0.00	Poor	No — worse than chance
0.01 – 0.20	Slight	No
0.21 – 0.40	Fair	Not yet
0.41 – 0.60	Moderate	Borderline — improve the rubric
0.61 – 0.80	**Substantial**	**Usually yes**
0.81 – 1.00	Almost perfect	Yes

THE GOTCHA

The Kappa Paradox

	Judge: PASS	Judge: FAIL
Human: PASS	**85**	5
Human: FAIL	5	**5**

They agreed on 90 of 100 answers.

p_o = **0.90** 😀

p_e = **0.82** (almost everything is "pass")

$\kappa = (0.90 - 0.82)/(1 - 0.82) =$ **0.44**

> 90% agreement → only "moderate" κ. When one class dominates, kappa gets harsh.

▸ **Load the "paradox" preset**

PUTTING IT TOGETHER

The judge-validation loop

1

Sample & label

Humans grade a real slice.

2

Run the judge

Same items, same rubric.

3

Measure κ

Agreement beyond chance.

4

Trust or fix

κ high → scale. Low → fix rubric, repeat.

> Once κ is high, the judge runs on thousands of items while the humans go home.

SINGAPORE · GOVERNANCE YOU CAN RUN

AI Verify & Project Moonshot

AI Verify — Singapore's **IMDA**, stewarded by the **AI Verify Foundation** (Google, IBM, Microsoft, Salesforce…): an open-source **testing framework + toolkit** checking AI against **11 governance principles**. **Testing ≠ certification** — it produces evidence, not a "safe" stamp.

AI VERIFY — 2022

Traditional ML

Classification & regression on tabular data. Fairness, robustness, explainability tests + governance process checks.

PROJECT MOONSHOT — 2024

Generative AI / LLMs

One of the first open LLM eval toolkits: benchmarking, red-teaming & safety baselines — and it uses an LLM judge.

Same foundation, different era: AI Verify for classic models, **Moonshot for the LLMs we build today**.

INSIDE MOONSHOT

And yes — it uses an LLM judge

A recipe = a **dataset** + a **metric**:

- **Connector** → talk to any LLM
- **Dataset** → prompts + targets
- **Metric** → score (incl. LLM-as-judge)
- **Recipe** → dataset + metric
- **Cookbook** → a themed set of recipes

- **Benchmarking** — capability, quality, trust & safety
- **Red-teaming** — automated adversarial prompts
- **Baseline testing** — a safety floor before you ship

▸ **Explore the Moonshot map**

THE 2025 FINDING

Risk is context-dependent

Singapore's **Global AI Assurance Pilot** (2025) ran the world's first technical testing of **real-world** GenAI apps — 17 organisations across 10 sectors.

The headline lesson:

> GenAI risk is specific — to your industry, use-case, language, culture and data.

A generic benchmark won't tell you if *your* app is safe. You still need **your own** domain evals.

FOR YOUR TEAM

Build your own eval harness

DATA

A real dataset

Representative prompts from your actual use-case.

JUDGE

A rubric'd judge

Clear criteria, structured output, bias guards.

CHECK

An agreement gate

Validate the judge with κ before trusting it.

CI

A regression gate

Run it on every change; block drops in score.

> You don't need a national programme — you need these four boxes wired together.

REMEMBER THIS

Six takeaways

- Open-ended outputs need **judgement**, not string-matching.
- **LLM-as-a-judge** gives human-like grading at machine speed — if you write a real rubric.
- Judges are **biased** (order, length, self-preference). Design around it.
- Never trust a judge you haven't **validated against humans** — and use **Cohen's Kappa**, not raw %.
- Risk is **context-specific**. Build **your own** evals; let AI Verify / Moonshot inspire the structure.
- Evals aren't a one-off: **wire them into CI**, curate the golden set, and **red-team** before you ship.

GO DEEPER

Resources & the live demos

THIS KIT

The interactive labs →

Judge walkthrough · Kappa calculator · Moonshot map

SINGAPORE

Official sources

aiverifyfoundation.sg · github.com/aiverify-foundation/moonshot · assurance.aiverifyfoundation.sg

Now go measure something. Thank you.

APPENDIX · CITATIONS

Sources & further reading

Singapore AI Verify / Moonshot

- `aiverifyfoundation.sg` — framework, the 11 governance principles, Project Moonshot
- `github.com/aiverify-foundation/moonshot` — the open-source LLM evaluation toolkit
- `assurance.aiverifyfoundation.sg` — Global AI Assurance Pilot (2025)

LLM-as-a-judge

- Zheng et al. 2023, *Judging LLM-as-a-Judge with MT-Bench & Chatbot Arena* — arXiv:2306.05685
- *Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge* (2024) — arXiv:2410.02736

APPENDIX · FOR THE EVAL NERDS

Fine print & honest caveats

- **Reference-guided** grading is really a *variant* of single-answer grading (the judge is handed a gold answer) — not a separate third axis.
- **Beyond Cohen: weighted κ** gives partial credit on *ordered* labels (1–5 stars); **Fleiss' κ** extends to a panel but is *nominal-only* with fixed raters; **Krippendorff's α** is the most general (any scale, missing data).
- **Formatting / markdown bias** is documented in *later* work — not the original MT-Bench paper.
- **Landis–Koch bands are arbitrary** (the authors said as much). Always read κ *next to* the confusion matrix — remember the paradox.